CODE :-

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

void addJob()
{
   int job;

   if (rear == MAX - 1)
   {
      printf("Queue Overflow! Queue is full.\n");
      return;
   }

   printf("Enter Job ID: ");
   scanf("%d", &job);

   if (front == -1)
      front = 0;

   rear++;
   queue[rear] = job;

   printf("Job added successfully.\n");
}
```

```
void deleteJob()
{
   if (front == -1 || front > rear)
   {
      printf("Queue Underflow! No jobs to process.\n");
      return;
   }

   printf("Processed Job: %d\n", queue[front]);

   if (front == rear)
   {
      front = -1;
      rear = -1;
   }
   else
   {
      front++;
   }
}

void displayJobs()
{
   int i;

   if (front == -1)
   {
      printf("Queue is empty. No pending jobs.\n");
      return;
   }
```

```
    printf("Pending Jobs:\n");

    for (i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }

    printf("\n");
}

int main()
{
    int choice;

    do
    {
        printf("\n JOB QUEUE SIMULATION  \n");
        printf("1. Add Job\n");
        printf("2. Delete/Process Job\n");
        printf("3. Display Pending Jobs\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                addJob();
                break;
```

```
            case 2:
                deleteJob();
                break;

            case 3:
                displayJobs();
                break;

            case 4:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice! Please try again.\n");
        }

    } while (choice != 4);

    return 0;
}
```

Output :-

```
JOB QUEUE SIMULATION
1. Add Job
2. Delete/Process Job
3. Display Pending Jobs
4. Exit
Enter your choice: 1

Enter Job ID: 101
```

Job added successfully.

Enter your choice: 1

Enter Job ID: 102

Job added successfully.

Enter your choice: 1

Enter Job ID: 103

Job added successfully.

Enter your choice: 3

Pending Jobs:

101 102 103

Enter your choice: 2

Processed Job: 101

Enter your choice: 3

Pending Jobs:

102 103

WRITEUP :-

Assignment no:- 9

Problem Statement :-
Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write a C program for simulating a job queue. Write functions to add a job and delete a job.

Objective :-
1) Understand the concept of Queue and FIFO.
2) Implement a linear queue using an array.
3) Implement functions to add and delete jobs.
4) Simulate processing of jobs in the order they enter the system.
5) Analyze the time and space complexity of queue operations.

Implementation :-

1) Platform :-
- 64 - bit Open Source Linux or its derivatives.
- Open Source C Programming tool like gcc/Eclipse editor

2 For i = front to rear
Print Queue[i]

D Main Program

MAIN()
1. Initialize front = -1, rear = -1
2. Repeat
Display menu
Accept choice

If choice == 1
Accept job
ENQUEUE (job)

Else if choice == 2
DEQUEUE()

Else if choice == 3
DISPLAY()

Else if choice == 4
EXIT

Else
Print "Invalid choice"

3. Until choice == 4
4. Stop

Time Complexity :-

→ Operations	Time Complexity
1) Enqueue	O(1)
2) Dequeue	O(1)
3) Display	O(n)
4) Check Overflow	O(1)
5) Check Underflow	O(1)

Conclusion :-

The job queue simulation demonstrates the FIFO principle of a linear queue. Jobs are added at the rear and processed/deleted from the front in the same order in which they enter the system.

FAQs :-

1) What is a Queue ?
=> It is a linear data structure in which insertion takes place at the rear and deletion takes place at the front

2) What is FIFO ?
→ It stands for First In, First Out. The element inserted first is the first one to be removed.

2) Test cond" :-

Test Case	Input	Output
TC01	Add one job	Job added successfully
TC02	Add multiple job	Job stored in FIFO order
TC03	Delete a job	First entered job is deleted
TC04	Delete from empty queue	Queue Underflow message
TC05	Add job when queue is full	Queue Overflow message
TC06	Display pending jobs.	All jobs displayed from front to rear

Pseudo Code :-

Step-1 Initialize
. Set front = -1 and rear = -1
. Declare an array queue [MAX] to store jobs

Step 2 Display Menu
. Add job
. Delete / Process job
. Display Pending jobs
. Exit.

Step-3 Perform Operations
A) Add job (Enqueue)
1. If rear == MAX - 1
Print "Queue Overflow"
Return

2. If front == -1
front = 0

3 rear = rear + 1

4 queue [rear] = job

5. Print "Job added successfully"

B) Delete job. (Dequeue) :-

DEQUEUE()
1. If front == -1 OR front > rear
Print "Queue Underflow"
Return

2. Print "Processed Job:", queue [front]

3. If front == rear
front = -1
rear = -1
Else
front = front + 1

C) Display Pending jobs.

DISPLAY()
1. If front == -1
Print "Queue is empty"
Return

3) Why is a queue suitable for an Operating system job queue?
→ It is suitable because jobs can be processed in the order in which they arrive, ensuring fairness when no priorities are used.

4) What is Queue overflow and Queue Underflow?
→ Queue Overflow:- Occurs when an attempt is made to insert an element into a full queue.
Queue Underflow:- Occurs when an attempt is made to delete an element from an empty queue.

5) What are the applications of Queue?
→ CPU and process scheduling, Printer job scheduling, Network data buffering, etc.

6) What is Circular Queue?
→ A circular queue is a queue in which the last position is connected to the first position, allowing previously freed spaces to be reused.

7) Why is Circular Queue preferred over linear Queue?
→ It makes better use of available memory by reusing empty spaces created after deletion, reducing wasted memory capacity.

8) What principle does a Circular Queue follow?
→ It follows the FIFO principle, just like a linear queue.

9) What are the basic operations performed on a Circular queue?
→ . Enqueue
. Dequeue
. Peek
. Display.

10) What are Overflow and Underflow cond" in a Circular Queue?
→ . Overflow:- Occurs when the circular queue is full, generally when (rear+1)%MAX == front.
. Underflow:- Occurs when front == -1, indicating that the queue is empty.

9/10